

历史平行线 – Code Review

我深度阅读了整个项目。整体印象是这是一个完成度相当高、工程质量不错的项目：146 个测试全部通过、`tsc --noEmit` 零错误、类型系统严密、`mockData` 已经做了轻量化拆分（`regions` 独立、`_stats.json/_featured.json` 预计算）、`LocaleProvider` 还做了 `context` 拆分来减少 `re-render`。这些细节说明作者有 React 性能意识。

但深入 review 后，有一批真实且重要的问题，下面按严重程度分级。

Critical – 必须修复

1. 后台管理系统完全没有鉴权，且整个审核/发布流程是“假的”

这是最严重的问题，分两层：

(a) `/admin/*` 路由零鉴权。全项目没有 `middleware.ts`、没有任何 `getSession/auth()/session` 检查。`src/app/admin/layout.tsx` 是个普通 `server component`，直接渲染后台界面。任何人访问 `/admin`、`/admin/review/events` 都能看到全部数据，包括 `imported / needs_review / rejected` 的草稿。

(b) 审核操作只改本地 **React state**，从不上报服务器。这是真正危险的——它会让运营者误以为自己发布了数据，实际什么都没发生：

AdminReviewEventsClient.tsx:61-66 的 `updateStatus`、83-90 的 `bulkUpdateStatus`、68-72 的 `updateScore`、97-104 的 `commitScoreEdit`，全部只调 `setEventsData(...)` 然后弹一个绿色 toast `"Status updated to \"published\""`。没有 **fetch**、没有 **server action**、没有写文件、没有 `revalidatePath`。刷新页面，所有改动消失。AdminImportsClient.tsx:50-72 的"开始导入"更夸张——是 `setTimeout` 配 `Math.floor(Math.random()*50)+10` 伪造出来的。

一个无鉴权、操作无持久化的后台 = 数据完整性的定时炸弹。

建议：

- 立即加 `src/middleware.ts`，对 `/admin/*` 做 cookie/session 校验（即便是临时的 Basic Auth 也比现在好）。
- 把 `updateStatus/bulkUpdateStatus/updateScore` 改成真正的 server actions 或 fetch 到一个 POST 路由，落库或写回 JSON + `revalidatePath`。
- 在鉴权就位之前，至少在 toast 文案里如实标注"仅本地预览"，不要骗用户。

2. 未发布数据通过公开 API 泄露

`src/app/api/data/people/route.ts:19,33-37:`

```
const publishedOnly = searchParams.get('published') !== 'false';
if (publishedOnly) { items = people.filter(p => p.dataStatus === 'published'); }
else { items = [...people]; } // ← 匿名即可拿全部
```

任意人 GET `/api/data/people?published=false` 就能 dump 出所有人物（含草稿/被拒）。`status=` 过滤（line 40）同理。admin review client 正是用这个口子拿数据的（AdminReviewPeopleClient.tsx:85）。

建议： `published=false / status!=published` 分支必须 gated by 鉴权；
否则直接删掉这个分支，让 admin 走 server action 拿数据。

3. 整个后台模块无 i18n，中文用户看到的却是全英文界面

`AdminDashboardClient / AdminLayoutClient / AdminImportsClient / AdminReviewPeopleClient / AdminReviewEventsClient` 全是硬编码英文
("Dashboard"、"Review: People"、"Publish"、"Needs Review"、"Reject All"...)。考虑到本站主体是中文历史、默认 locale 是 zh-CN，这是个明显的体验断层。i18n 体系 (`dictionaries/zh.ts + useLocale`) 已经存在，只是 admin 没接。

High – 应尽快修

4. 客户端组件缺 'use client' 指令（隐患）

`src/components/parallel/RegionColumn.tsx`、`src/components/cards/PersonCard.tsx`、`src/components/cards/EventCard.tsx` 都调用了 `useLocale()` (client hook)，但文件顶部没有 'use client'。现在能跑只是因为它们恰好被某个 'use client' 的父组件间接拉进了 client bundle。一旦将来有 server component 直接 import 它们，会编译失败或运行时抛 `useLocale must be used within a LocaleProvider`。补上 'use client' 即可。

5. TimeRangeSlider 拖拽抖动 + 除零风险

src/components/parallel/TimeRangeSlider.tsx:76-106 的 useEffect 把 minVal, maxVal 放进了依赖数组——而这两个值在每次拖拽移动时都会变，于是**每次 mousemove 都会 teardown+重绑四个 window listener**，导致拖动卡顿/丢事件。应改用 ref 读最新值，依赖只留 dragging。

同文件 39-52：当 events 为空或全部 startYear 为 undefined 时，minYear === maxYear === 0, range === 0, valueToPercent 算出 NaN%。父组件 ParallelTimelineView 有 guard，但这个 slider 自己没有，需加 range === 0 防御。

6. PersonSelector 每次 add/remove 都重新发请求

src/components/compare/PersonSelector.tsx:32-59: debounce effect 的依赖里有 selectedIds（一个 useMemo 重建的 Set），用户每勾选/取消一个人都会对同一查询重新发一次请求。依赖应换成 query + 一个稳定派生量（如 size 或 join 的 id 串），并且加请求序号/AbortController 防止竞态（目前 .then 里没有 cancelled 检查，快速打字会显示过期结果）。

7. VerticalCompareTimeline.fmtYear 对英文用户显示中文

src/components/compare/VerticalCompareTimeline.tsx:33-36:

```
function fmtYear(y) { if (y < 0) return `前${Math.abs(y)}`;
return String(y); }
```

整个纵向对比时间轴对 en 用户也显示 前500。应走 t() 或 formatYearLabel。

8. 多处向客户端打包了完整的数据集

`src/app/compare/page.tsx` 把全部 `publishedEvents` 数组作为 `prop` 传给 `ComparePageClient`（后者只在客户端按选中人物过滤）；`relationships/page.tsx` 同样把完整 `relationships` 数组 + `Map` 打到客户端；`admin/review/events/page.tsx` 把含草稿的全部 `events` 打到客户端。这些都会进 `RSC payload / client bundle`。项目其它地方已经很小心地把 `mockData` 挡在 `client` 之外（值得肯定），这几处是漏网的。应改成按需 `fetch` 或 `server` 端先过滤。

9. `notFound()` 在 `client` 组件里调用 + 全站没有 `generateMetadata`

`PersonPageClient.tsx:27`、`EventPageClient.tsx:33` 在 `'use client'` 组件里调 `notFound()`。Next.js 里 `notFound()` 从 `client` 抛出并不保证触发 `not-found.tsx` 边界（会落到最近的 `error boundary`）。更干净的做法是在 `server` 端 `people/[id]/page.tsx` / `events/[id]/page.tsx` 里判断 `!person` 时 `notFound()`。

另外全项目 `grep` 不到任何 `generateMetadata`，动态路由 `/people/[id]`、`events/[id]` 全用 `layout.tsx` 里那一个全局 `title`，SEO/社交分享基本失效。

10. 并行页/admin 的 `fetch` 没有请求取消

`src/app/parallel/ParallelPageContent.tsx:50-60` 和 `AdminReviewPeopleClient.tsx:73-99` 的 `fetch effect` 都没有 `AbortController/cancelled flag`（而 `SearchResults.tsx:92-101` 做对了）。快速切换年份/筛选时，旧响应可能覆盖新响应——经典竞态。

🟡 Medium – 值得优化

11. i18n 不一致：大量内联 locale === 'en' ? ... : ... 绕过了 t() 和 toScript()

项目有完整的 t 字典和 toScript (zh→zh-TW 转换)，但很多组件绕过它，直接写三元：

- CivilizationTimeline.tsx、EventPreviewPanel.tsx、EventMapView.tsx、VerticalCompareTimeline.tsx、CompareTimeline.tsx、EventCard.tsx、PersonSelector.tsx 等十几处。
- ParallelWorldView.tsx:15、SearchBox.tsx:8、EmptyState.tsx:5-6、SourceList.tsx:7、TimeRangeSelector.tsx:7-10 更严重——这些组件连 locale 都没读，直接硬编码中文，英文用户也会看到中文空状态/placeholder。
- VerticalCompareTimeline.tsx:299 写了 t.compare.mainPerson || '主体'——字典明明有 mainPerson 还要 fallback 到硬编码 '主体'，且这个 fallback 不走 toScript，zh-TW 用户会看到简体。

12. 把中文 tag 字符串当 i18n key

CompareTimeline.tsx:86-88、VerticalCompareTimeline.tsx:405,673: event.tags.includes('出生') / '逝世' / '成就'。把展示文案耦合到 tag 的中文取值上，一旦 tag 被本地化或换成 'birth' 就静默失效。tag 应当是 enum key。

13. 可访问性 (a11y) 大面积缺失

- `CivilizationTimeline.tsx:194, 267, 296`、`EventMapView.tsx:355-371`: 可点击的 `<text>/<circle>/<rect>` 没有 `tabIndex`、`role="button"`、`onKeyDown`, 键盘用户和屏幕阅读器完全够不到。
- `TimeRangeSlider.tsx:145-170`: 自定义双滑块只有 `onMouseDown/onTouchStart`, 没有 `role="slider"`、`aria-valuemin/max/now`、没有方向键支持。(可参考本项目的 `VerticalCompareTimeline`, 它正确做了 `role="button"` `tabIndex={0}` `onKeyDown`。)
- `VerticalCompareTimeline.tsx:763-864` 的模态框有 `role="dialog"` `aria-modal` 和 `Esc` 处理 (👍), 但没有 **focus trap**、打开时不把焦点移入、关闭时不还原焦点, `Tab` 会跑到背景内容。
- `EventMapView.tsx:431`、`CivilizationTimeline.tsx:345` 的关闭按钮 `×` 没有 `aria-label`。
- `EventPreviewPanel.tsx:53` 的关闭按钮 `aria-label="Close"` 是硬编码英文。

14. 性能: memo 被内联函数打败 + 死代码

- `ParallelTimelineView.tsx: nodeSize/nodeColor/getXPercent (line 64/70/76)` 每次 `render` 都是新函数引用, 传给 `memo()` 的 `TimelineTrack`, `memo` 永远短路不了。应 `useCallback` 或提到组件外。
- `ParallelTimelineView.tsx:38`: `allEvents` 的 `useMemo` (含一次全量排序) 从未被使用 (`mobile` 视图直接 `map groups`), 还加了

eslint-disable no-unused-vars 掩盖。这是每次 render 都跑的死代码，直接删。

- CivilizationTimeline.tsx:99-104: ticks 数组、以及 render 内对每个 civ 做 `slice(0,civIndex).reduce(...)` 累加高度，都没 memo，每次 hover (`setHoveredPeriod`) 都重算。`onMouseMove` (235) 每像素一次 `setState` 也应该 RAF 节流。
- EventMapView.tsx:152,321: `allMarkers.find()` / `markers.some()` 套在 `.map` 循环里，是 $O(n^2)$ ；预先建 `Map<id,marker>/Set<id>` 即可 $O(n)$ 。
- VerticalCompareTimeline.tsx:165-213: `layoutCompareEvents` 是 `useCallback`，但 `useCallback` 只缓存函数、不缓存结果——每次 render 对每个对比人物都重新算一遍布局。应改成 `useMemo` 一次性算出所有人的布局。另外 line 201-207 算了 `bestLane/lastYPerLane` 但没写进返回的 `Placed` (只有 `event/y`)，整段 lane 逻辑是 no-op——要么是漏了 `lane offset`，要么该删。

15. RelationshipsPageClient 的力导向布局在多线程跑 $O(100 \times n^2)$

`RelationshipsPageClient.tsx:66-118` 的 `runForceLayout` 是同步 100 次迭代 $\times n^2$ ，节点一多会卡 UI。考虑 Web Worker 或 `requestIdleCallback`。另外 488 的 `regionMap.get(...)`！非空断言有崩溃风险（传入的 `regionMap` 只含相关人物，选中人物的 `region` 可能不在里面）。

Low / 信息性

- `npm run lint` 命令本身坏了: `Invalid project directory .../历史平行线/lint--next lint` 把 `lint` 当成了目录参数。
`package.json` 的 `"lint": "next lint"` 需要确认 Next 16 的正确调用方式 (可能是 `next lint --dir src` 或已废弃, 需迁移到 ESLint 直接调用)。这意味着项目目前实际没有有效的 **lint** 流水线, 上面发现的 `hooks deps` 等问题本来 `exhaustive-deps` 就能抓到。
- `CompareTimeline.tsx:143` 用 `BC${year}` (无空格、BC 而非 BCE), 和全站 `BCE/前xxx` 不一致。
- `ComparePageClient.tsx:28-34` 的 `fetch().then(r=>r.json())` 没检查 `r.ok`, `4xx/5xx` 时 `data.items` 为 `undefined`, 用户无任何错误反馈。
- `compute-stats.mjs:91` 把 `totalRegions: 34` 硬编码进统计, 而非从 `regions` 实算——数据漂移隐患。
- `admin/review/people` 的 `debounced search effect (61-70)` 和 `status/region change effect (102-105)` 都会 `setPage(1)`, 搜索框改动会触发两次 `setPage` + 双 `fetch`。
- `not-found.tsx` 是 `'use client'` (可接受, 但配合 `finding #9` 一起看)。
- `jest.config.js` 在本次改动之列 (`git status M`), 但 146 测试全绿, 👍。

架构层面的总体评价

做得好的地方 (值得保留) :

- 核心库 (`date.ts/parallel.ts/search.ts/validation.ts/dataProvider.ts`) 职责清晰、纯函数、可测——146 个单测覆盖了 `normalization/datePrecision/confidence/dedupe/parallel/search`, 质量很高。
- 数据层的轻量化拆分思路 (`regions.ts` 独立、`_stats.json/_featured.json` 预计算、`server-actions.ts` 用 `'use server'` 把 `mockData` 挡在服务端) 方向完全正确, 注释里甚至解释了"避免 117MB load", 看得出有过性能调优。
- `LocaleProvider` 把 `locale` 和 `translation` 拆成两个 `context` 来减少 `re-render`、`opencc-js` 懒加载、`useLocaleOnly` 优化——这是有经验的 React 写法。

架构上最大的问题:

项目正处于"从静态 `mockData` 迁移到真正可写数据层"的过渡期, 但写路径完全没建起来 (`finding #1/#2`), 而读路径又过早地把未鉴权的写能力暴露成了公开 API (`finding #2`)。这两个加在一起, 是当前代码库里风险最高的区域。建议把这当作下一个迭代的头等大事: 定一个持久化方案 (`SQLite/Postgres/甚至写回 JSON 文件 + git`), 然后让 `admin` 操作真正落库、API 的非 `published` 分支加鉴权。

优先级建议 (按这个顺序修)

1. **#1 后台鉴权 + 真实持久化** (或至少先关掉后台入口 + 改 `toast` 文案诚实告知)
2. **#2 收紧 `/api/data/people` 的 `published=false / status` 分支**
3. **#3 admin 接入 i18n**
4. **#5/#6/#10 滑块抖动、`selector` 重复请求、`fetch` 竞态** (用户可感知的 bug)

5. **#4** 补 `'use client'`
6. **#8** 收敛传给 **client** 的全量数据集
7. **#11/#12 i18n** 统一走 `t()/toScript`
8. **#14** 性能死代码 + **memo** 失效
9. **#13 a11y**
10. 修 `npm run lint` (修好后, 很多 hooks 问题会自动浮现)

需要的话, 我可以直接动手修其中任意一条——建议从 **#1** (后台鉴权) 或 **#5** (滑块抖动, 改动小、收益明显) 开始。你想先动哪个?